Artificial Intelligence (CSL7610) Assignment

Debmalya Bhattacharya
Roll No. `P26CS0002`

1 Problem Formulation

Given a knowledge base KB (facts and rules in natural language) and a query q, the task is to output

$$\hat{y}(\mathrm{KB}, q) = \begin{cases} \textsc{True} & \text{if KB} \models q, \\ \textsc{False} & \text{otherwise.} \end{cases}$$

The label is binary. FOLIO's three-way labels are mapped accordingly: *False* and *Uncertain* both become False, because in neither case is the query a logical consequence of the KB. RuleTaker's `flag` is already True/False, and CounterBench's *yes*/*no* answers map to True/False.

The three benchmarks test different kinds of reasoning, and this shapes the design:

- **RuleTaker** (`jise/ruletaker`) — synthetic English knowledge bases split into a *Facts:* and a *Rules:* section. Facts and rules are ground (about named entities such as *the river* or *Emma*), and each if–then rule is written in one of several paraphrases (*"Assuming A, we can say B"*, *"Based on A, it is evident that B"*, ...); proofs of depth 1–5.
- **FOLIO** (`yale-nlp/FOLIO`) — expert-written, open-domain English with full first-order structure (quantifiers, disjunction, exclusive-or, biconditionals). Each story comes with a first-order logic (FOL) annotation.
- **CounterBench** (`CounterBench/CounterBench`) — deterministic causal stories with nonsense variable names (e.g. *Ziklo causes Blaf*), queried with counterfactual questions (rung 3 of Pearl's ladder of causation).

Because no single logic covers all three well, the system is a *router* over three representations, each paired with the inference procedure that is sound and efficient for it (Figure 1).

Input	**→ Parser**	**→ Representation**	**→ Inference**
RuleTaker text	→ rule-template parser	→ ground literals + Horn rules	→ stratified forward chaining
FOLIO FOL	→ recursive-descent FOL parser	→ CNF clauses	→ resolution refutation
CounterBench text	→ causal-relation parser	→ boolean structural equations	→ abduction + do-calculus

Figure 1: System overview. A router selects the engine from the fields present (FOL annotation ⇒ resolution; causal question ⇒ counterfactual engine; *Facts:*/*Rules:* sections ⇒ forward chaining).

2 Representation

2.1 RuleTaker: ground literals and Horn rules

Every sentence under *Facts:* becomes a ground *literal* $\ell = (s, a)$, where $s \in \{+, -\}$ is the sign and a a normalised atom. Every sentence under *Rules:* becomes a Horn rule $\ell_1 \wedge \cdots \wedge \ell_k \Rightarrow \ell_0$. Atoms are normalised so that the same proposition always gets the same name: the text is lower-cased, articles are dropped, every word is lemmatised ("plays" → `play`, "is" → `be`) and a negation word (*not*, *n't*, *never*) is removed and recorded in the sign.

A rule sentence is split into an antecedent and a consequent by a small set of templates: an antecedent cue (*if*, *assuming*, *given that*, *based on*, *when*, *since*, ...) followed by a comma, or the forms "B if A" and "A implies B". Consequent cues (*then*, *we can say*, *it is evident that*, *it follows*

Sentence	Representation
The train plays instruments.	+ train play instrument
The pig is an artist.	+ pig be artist
Assuming the river dances gracefully, we can say the river is fast.	river dance gracefully ⇒ river be fast
Based on charlie is fast, it is evident that stanford university volunteers.	charlie be fast ⇒ stanford university volunteer
Given that the globe is colorful, it follows that mike is colorful.	globe be colorful ⇒ mike be colorful
When emma makes pizza, we can conclude alice is an artist.	emma make pizza ⇒ alice be artist
If the globe is friendly, then the dog runs marathons.	globe be friendly ⇒ dog run marathon

that, *we can conclude*, ...) are stripped. An antecedent with *and* becomes a multi-literal body; one with *or* becomes one rule per disjunct. Over all 1,625 public RuleTaker queries the parser read 26,180 rule sentences and left **none** unparsed.

Why this representation. In `jise/ruletaker` every fact and rule is about named entities, with no variables and no function symbols, so the knowledge base is a *propositional* Horn program (with negation-as-failure if a negated literal appears). This gives (a) a finite set of atoms, so forward chaining always terminates, and (b) an exact encoding: the only source of error left is the translation, which is template-based because the surface syntax is template-generated. Lemmatisation makes "the cat plays instruments" in a rule match "The cat plays instruments." in the facts regardless of article or inflection. For other RuleTaker formats with variables (*"If someone is young then they are smart"*) the notebook keeps a first-order parser that produces rules with a variable, which are run by the same engine.

2.2 FOLIO: clausal first-order logic

FOLIO computes FOL formulas with the connectives $\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \oplus$ and the quantifiers $\forall, \exists$. It uses a precedence climbing parser to parse the formulas into an abstract syntax tree (preference $\neg > \wedge > \vee > \oplus > \rightarrow > \leftrightarrow$, $\rightarrow$ is right-associative). Then the abstract syntax tree is transformed to conjunctive normal form (CNF) in four steps:
1. elimination of $\leftrightarrow$, $\oplus$ and $\rightarrow$, where $A \oplus B \equiv (A \vee B) \wedge (\neg A \vee \neg B)$;
2. moving negations inside the formula until negation normal form is obtained;
3. Skolemisation: every $\exists y$ within the scope of $\forall x_1 \ldots x_k$ is substituted by a new function $f(x_1, \ldots, x_k)$;
4. distributive transformation of $\vee$ over $\wedge$ and removal of tautological clauses.

Reasons for this representation. The FOLIO database has actual disjunction, existentials, and exclusive-or, none of which can be formulated using Horn clauses. Clausal form is the form required by the resolution procedure, which is *refutation-complete* for first order logic. The FOL annotations provided along with the database are taken as the target of the translation. This is an intentional choice, since open domain translation of English to FOL is unsolved, and doing so using templates would only shift the locus of errors from the reasoner to the parser.

2.3 CounterBench: deterministic structural causal model

Causal sentences become boolean structural equations; each variable that is caused by something becomes *endogenous*:

$$\text{"}\textit{Blaf and Ziklo together cause Trune}\text{"} \implies \text{Trune} \coloneqq \text{Blaf} \wedge \text{Ziklo},$$
$$\text{"}\textit{Trune causes not Vork}\text{"} \implies \text{Vork} \coloneqq \neg\, \text{Trune}.$$

If several sentences cause the same variable, their right-hand sides are OR-combined. Sentences of the form *"We observed V"* become evidence $V{=}1$. Questions become interventions:

- basic/conditional — *"Would Y occur if not X instead of X?"* $\Rightarrow$ factual $X{=}1$, intervention $\mathrm{do}(X{=}0)$;
- joint — *"... if not X and not V?"* $\Rightarrow \mathrm{do}(X{=}0, V{=}0)$;
- nested — *"Assume not X ... further suppose not V. Would Y occur?"* $\Rightarrow$ the same joint intervention.

Why this representation. CounterBench is generated from deterministic SCMs $M = \langle U, V, F \rangle$, so a boolean equation per variable is an exact encoding rather than an approximation. Counterfactual queries cannot be answered by entailment over the stated sentences alone, because they require *replacing* equations (the do operator). Making the equations explicit allows the textbook three-step counterfactual procedure to be applied directly (Section 3.3).

3 Reasoning Algorithms

3.1 Stratified forward chaining (RuleTaker)

Facts are stored as a set of signed atoms. A rule fires when all of its body literals hold; each derived fact stores the rule that produced it, which yields the proof traces in Section 5.

Negation. RuleTaker is built under a *closed-world assumption* (CWA): "Bob is not big" is true when $\mathsf{big}(\mathsf{bob})$ cannot be derived. This is negation-as-failure (NAF), which is non-monotonic: a NAF test that succeeds early may become false once more facts are derived. To make the result well-defined, predicates are *stratified*. We compute a level $\sigma(p)$ for each predicate with

$$\sigma(h) \;\geq\; \sigma(b) \;\text{ for a positive body literal } b, \qquad \sigma(h) \;\geq\; \sigma(b) + 1 \;\text{ for a negated body literal } b,$$

and evaluate the strata in increasing order, running each to a fixpoint. A NAF test therefore only consults predicates that are already complete, and the computed model is the program's unique perfect model. An *open-world* (OWA) mode, in which $\neg p$ must be derived explicitly, is also implemented. The mode is not hard-coded: it is *selected on the RuleTaker train split*. CWA was selected, which confirms that the dataset follows closed-world semantics.

Algorithm 1 Stratified forward chaining with negation-as-failure

Require: facts F, rules R, query literal q

compute strata σ from the dependency graph of R
for each stratum k in increasing order **do**
  repeat
    for each rule $(B \Rightarrow h) \in R$ with $\sigma(h) = k$ **do**
      for each substitution θ unifying the positive literals of B with F **do**
        if no $\neg b \in B$ has $b\theta \in F$ **then** ▷ NAF, lower strata
          $F \leftarrow F \cup \{h\theta\}$; record justification
        end if
      end for
    end for
  until no new fact is added
end for
return $q \in F$ if q is positive, else $\bar{q} \notin F$

Why forward chaining rather than backward chaining. Each RuleTaker KB is small (a few facts and a few dozen rules), so its fixpoint is reached in a handful of passes, and after that the query is a set lookup. Forward chaining also makes NAF straightforward to implement correctly, which is harder in a top-down prover with loop checking. The measured cost is 0.45 ms per query.

3.2 Resolution refutation (FOLIO)

To decide KB $\models q$, we test whether KB $\wedge \neg q$ is unsatisfiable. Both are converted to CNF and a *given-clause* loop searches for the empty clause $\square$:

- **Set of support:** only clauses that descend from $\neg q$ are selected as the given clause, so the search stays focused on the query instead of saturating the KB.
- **Best-first selection:** a priority queue ordered by clause weight ($10\times$ the number of literals, plus term depth). This is a uniform-cost search over the space of derivable clauses, preferring short clauses, which are closer to $\square$.
- **Unification:** most-general unifiers with occurs check; variables are renamed apart before each resolution step.
- **Redundancy control:** factoring, deletion of tautologies, elimination of variant clauses via canonical renaming, and a cap of 8 literals per clause.
- **Budget:** 2 s and 4,000 clauses per query.

If the budget runs out before $\square$ is found, the prediction is FALSE.

Why resolution, and why a budget. Resolution with set of support is refutation-complete, so every true entailment has a proof. First-order validity is, however, only semi-decidable, so a query that is *not* entailed may never terminate. A resource bound is unavoidable. Returning FALSE on timeout is the natural default, because non-entailment (*False* or *Uncertain*) is the majority outcome in FOLIO. The consequence is that every timeout on a genuinely entailed query becomes an error; we quantify this in Section 4.

3.3 Counterfactual inference (CounterBench)

We follow Pearl's three-step procedure for counterfactuals:

1. **Abduction.** Find an assignment u to the exogenous variables U that is consistent with the observations and the factual premise. The search is breadth-first over the number of unobserved exogenous variables switched on: the default world, in which unobserved variables are absent, is tried first, then all worlds with one variable switched on, and so on. The first consistent world is the minimal one.
2. **Action.** Replace the equation of each intervened variable X by the constant $\mathrm{do}(X{=}x)$.
3. **Prediction.** Evaluate the remaining equations in topological order and read off Y.

Why this default. CounterBench represents a particular variable as 1 if the variable exists, and 0 if the variable does not exist. "Absent unless observed" prior exactly defines the difference between *conditional* problems and *basic* problems. In the example presented below, observation of Trune results in Lumbo. Breadth-first search finds the minimal world with this prior, and since there are at most nine variables in each story, exhaustive search is inexpensive (0.19 ms per query).

4 Experiments

4.1 Setup

All three datasets were loaded from Hugging Face with `load_dataset`, and **every public sample** was evaluated:

- **RuleTaker** (`jise/ruletaker`): train 1,000 + test 250 + ood_test 375 = 1,625 queries.
- **FOLIO** (`yale-nlp/FOLIO`, gated; accessed after accepting its terms and logging in): train 1,001 + validation 203 = 1,204 queries.
- **CounterBench** (`CounterBench/CounterBench`): the question files `data_balanced_alpha_V1.json` (1,000) and `data_balanced_backdoor_V2.json` (200) = 1,200 queries. The files are loaded one at a time, because the repository also holds `meta_model_*.json` files with a different schema that break a plain `load_dataset` call.

The system has no trainable parameters. The only data-driven choice, closed- versus open-world negation for RuleTaker, was made on the RuleTaker *train* split; accuracy is also reported per split, so the RuleTaker test and ood_test numbers are untouched by that choice. The `reasoning` column of RuleTaker, which contains the gold proof, is never read. Timing covers inference only, in single-threaded Python.

4.2 Results

Table 1 reports accuracy and inference time per dataset. The *Overall Inference Time* is computed as the assignment specifies: the average time per sample is computed within each dataset (last column), and these three averages are then averaged over the three datasets, giving $(0.000449 + 0.112616 + 0.000191)/3 = \mathbf{0.0378\,s\ per\ sample}$. Because each dataset counts equally, this figure is dominated by FOLIO; the micro-average over all 4,029 queries is 33.89 ms per query. Table 2 breaks accuracy down by split and by question type.

Dataset	N	**Accuracy (%)**	**Time (s)**	**ms / query**	**Avg. s / sample**
RuleTaker	1625	98.46	0.73	0.45	0.000449
FOLIO	1204	81.15	135.59	112.62	0.112616
CounterBench	1200	90.42	0.23	0.19	0.000191
Overall	**4029**	**90.89**	**136.55**	33.89	
Overall Inference Time (mean of the per-dataset averages)					**0.0378**

Table 1: Accuracy and inference time on all public samples. Overall accuracy is micro-averaged over all queries. Overall Inference Time is the average time per sample in each dataset, averaged over the three datasets.

By split				**By depth / question type**		
Dataset	Split	N	Acc. (%)	Dataset	Group	Acc. (%)
RuleTaker	train	1000	98.30	RuleTaker	depth-1	100.0
	test	250	97.60		depth-2	96.4
	ood_test	375	99.47		depth-3	99.0
FOLIO	train	1001	80.52		depth-4	100.0
	validation	203	84.24		depth-5	98.4
CounterBench	alpha_V1	1000	91.00	CounterBench	basic / conditional	92.0 / 90.0
	backdoor_V2	200	87.50		joint / nested	90.0 / 92.0
					backdoor	87.5

Table 2: Accuracy by split and by RuleTaker proof depth / CounterBench question type. The negation mode selected on the RuleTaker train split was CWA.

4.3 Analysis: why the numbers look like this

RuleTaker: the errors in the labels are unrelated to the proofs. RuleTaker gets 98.46% overall, 97.6% on the test split and 99.5% on the out-of-distribution split (Table 2), and performance remains high regardless of the proof depth: 100% on depth 4 and 98.4% on depth 5. This is unsurprising: the reasoning engine is sound and complete for stratified propositional Horn programs, and the parser parsed all 26,180 rule sentences without error, thus making the problem of long chains of rules as easy as that of short ones.

The mistakes made by the system we checked are due to cases where the gold label conflicts with a rule stated explicitly in the context. In Trace 2 (Section 5) the knowledge base has the fact *"The train plays instruments"* and the rule *"Assuming the train plays instruments, we can say the artist is fast"*, but the query *"The artist is fast"* is marked FALSE. A similar case is where from the *"Emma makes pizza"* and *"When emma makes pizza, we can conclude alice is an artist"* the query *"Alice is*

an artist" is labelled FALSE. The two queries above are clearly entailed by the given statements under the specified semantics, thus they cannot be answered negatively by any sound reasoner.

FOLIO: constrained by the semi-decidability budget and by the data. FOLIO is the dataset with the lowest scores (81.15%: 80.52% on train, 84.24% on validation) and consumes 99% of all runtime (135.59 out of 136.55 s). The reasons are one and the same. Many FOLIO conclusions use $\oplus$ and $\leftrightarrow$ nested into conditionals, like in the example in Section 5. The CNF transformation leads to the exponential increase in the number of clauses because already $A \oplus B$ translates into two clauses, and the size of the search space increases exponentially with the size of the proof. For such queries the prover runs out of the 2 s budget frequently. This is where the 113 ms average comes from, and each timeout is necessarily an error for entailed queries. Another, minor reason for the errors is the data. Some annotations of the FOLIO's FOL queries have been reported to be inconsistent with the gold labels, and there is no way to fix this with a prover that is true to the annotation. Increasing the budget allows one to sacrifice runtime for the sake of accuracy; but the evaluation rewards both factors, so we kept the 2 s limit.

CounterBench: semantics of underspecified variables. The basic, conjunctive and nested questions have their answers determined after parsing the equations, and all the worked examples checked yield correct results. The remaining 9.6% of errors can be accounted for by two factors: (i) alternative linguistic constructions that do not appear in the relation parser, especially in the second set of questions (backdoor questions, 87.5% vs 91.0% in the first set), which are worded differently from the first set; and (ii) handling of variables that are not caused nor observed. "Absent by default" semantics is an assumption, and in all cases where a story requires a different interpretation, abduction finds a different world than the intended one. Both points are part of the translation and the modelling assumptions, but not of the inference process. Since the abduction stage is exhaustive, the inference itself is precise for the model it is given.

Efficiency. Forward chaining and SCM evaluation are polynomial in the size of the KB, and run in under 0.5 ms per query. Resolution is the only component with worst-case exponential behaviour, which is why the design pairs it with set of support, best-first selection and a hard budget.

5 Example Reasoning Traces

Trace 1 — RuleTaker, forward chaining (predicted True)

```
Context: Facts: The globe is colorful. The globe is friendly. The globe is colorful. ...
         Rules: Given that the globe is colorful, it follows that mike is colorful. ...
Query:   Mike is colorful.
Rule parsed:  globe be colorful  =>  mike be colorful
Trace:
  mike be colorful  <= globe be colorful
    globe be colorful  [fact]
```

The consequent cue (*"it follows that"*) is stripped and the query is derived in one step.

Trace 2 — RuleTaker, a label that contradicts a stated rule (gold False, predicted True)

```
Facts: The train plays football. The artist is fluffy. The train plays instruments.
Rules: Based on the train plays instruments, it is evident that the cat plays instruments.
       Given that the train plays instruments, it follows that the mountain is fluffy.
       Based on the artist is fluffy, it is evident that the cat plays instruments.
       Given that the train plays football, it follows that the mountain plays football.
       If the train plays instruments, then the artist is fluffy.
       Assuming the train plays instruments, we can say the artist is fast.
Query: The artist is fast.
Parsed: 3 facts, 6 rules, 0 unparsed; forward-chaining steps to fixpoint: 4
Trace:
  artist be fast  <= train play instrument
    train play instrument  [fact]
=> True   (gold label: False)
```

The last rule applies directly to a given fact, so the query is entailed in one step. This is the characteristic RuleTaker "error" analysed in Section 4.

Trace 3 — FOLIO, resolution (gold False, predicted False)

```
Query: Bonnie performs in school talent shows often.
Resolution: KB clauses  +  negated query clause  ->  set of support saturated after 1 step,
            empty clause not derivable  =>  not entailed  =>  False
```

Saturation without $\square$ is a genuine proof of non-entailment within the explored clause space.

Trace 4 — FOLIO, budget exhaustion (gold True, predicted False)

```
Query: If Bonnie is either both a young child or teenager who wishes to further her
  academic career ... and chaperones high school dances or neither ..., then Bonnie is
  either a student who attends the school or is an inactive and disinterested member.
Resolution: status = timeout after 28,031 resolution steps  =>  False (wrong)
```

This is the characteristic FOLIO failure analysed above: an XOR nested inside a conditional expands into many clauses, and the proof is not found within the 2 s budget.

Trace 5 — CounterBench, abduction–action–prediction (gold yes, predicted True)

```
given_info: We know that Ziklo causes not Blaf, Blaf and Trune together cause Vork,
            Vork causes Lumbo. We observed Trune.
question:   Would Lumbo occur if not Ziklo instead of Ziklo?

Equations : Blaf := NOT Ziklo ;  Vork := Blaf AND Trune ;  Lumbo := Vork
Abduction : exogenous {Ziklo, Trune}; evidence Ziklo=1 (factual), Trune=1 (observed)
Action    : do(Ziklo = 0)
Prediction: Blaf = 1 -> Vork = 1 AND 1 = 1 -> Lumbo = 1   =>  yes
```

Without the observation, the default world would give Trune= 0 and therefore Lumbo= 0. The observation is exactly what changes the answer, which is why the "absent unless observed" prior is the right default for this benchmark.

6 Conclusion

Matching each benchmark with the logic needed for it results in an entirely classical and easily interpretable approach which is scored at 90.89% overall accuracy for all 4,029 public queries (RuleTaker 98.46%, FOLIO 81.15%, CounterBench 90.42%), and whose Overall Inference Time is 137 s (mean per dataset average is 0.0378 s per sample). Every prediction is justified by a symbolic reasoning: derivation tree, refutation or causal computation. Error analysis shows clear next steps, not the inference engines. In RuleTaker, the parser handles each rule sentence, and the rest are labels conflicting with rules. In FOLIO, adaptive time budgets and improved clause selection (hyper-resolution, unit preference) would decrease timeouts. In CounterBench, the relation parser has to be expanded to the second question phrasing.